Deep Reinforcement Learning for Walking Control of a Biped Robot: A Reduced Action Space via Trajectory-Level Control

An English Summary of a 2022 M.Sc. Thesis

Reyhaneh Sadat Hosseininejad[1]

[1]Center of Advanced Systems and Technologies (CAST), School of Mechanical Engineering, College of Engineering, University of Tehran, Iran

Abstract

This document is a condensed English summary of my Master's thesis in Mechanical Engineering (Applied Design), defended at the University of Tehran in early 2022 under the supervision of Dr. Aghil Yousefi-Koma and Dr. Masoud Shariat-Panahi. The original thesis is written in Persian (titled, in literal translation, *"Walking Control of a Legged Robot Using Reinforcement Learning Algorithms"*); this summary preserves its scientific content, methods, and results faithfully. The work studies the effectiveness of machine-learning methods, under a deep reinforcement-learning (RL) approach, for the online control of stable walking of a full-size humanoid robot — the *Surena IV* humanoid built at the University of Tehran — on flat ground. The study proceeds in three parts. First, several RL algorithms (PPO, DDPG, TD3, SAC) are compared, and the most suitable for the robot's dynamics — PPO — is selected. The chosen algorithm is then implemented under two complementary approaches, each its own RL environment: in the first, the agent's output is the torque or position of the robot's joint-actuating motors; in the second — the thesis's main methodological contribution — the agent's output is, instead, the changes of the center-of-mass and swing-foot trajectories, from which the joint angles are obtained by a closed-form inverse-kinematics step, reducing the dimensionality of the state and action spaces. Third, given the central role of the reward in RL, the effect of different robot-performance metrics on the reward is studied and a new reward function is proposed. On the Surena IV lower body (twelve degrees of freedom), the first approach produces motion that keeps balance throughout the episode (with no constraint on the gait shape), and the second succeeds up to roughly two forward steps (about 70 cm); recovery from near-unstable states is identified as the principal open challenge. All findings, including an observed reward-exploitation behaviour, are reported honestly and without embellishment.

Keywords: Walking control of a legged robot, humanoid robot, reinforcement learning, PPO algorithm.

Note on provenance and authorship. This is an English summary of my Master's thesis in Mechanical Engineering (Applied Design), completed at the University of Tehran in 2022. The original thesis is written in Persian and is available in full here: original Persian thesis (PDF). This summary was prepared in 2026 so that the work is accessible to an English-speaking and international audience; the translation and condensation were produced with the assistance of Claude (Anthropic, Claude Opus 4.8). All scientific findings, numbers, methods, and limitations are reported as they appear in the original thesis, kept intact and honest, and reflect what was achievable and known in 2022.

1 Introduction

Bipedal humanoid locomotion is among the hardest problems in robotics: the system is high-dimensional, underactuated, and only intermittently in contact with the ground, so balance must be actively maintained at every instant [1]. Classical approaches to humanoid walking rely on reduced-order dynamic models and explicit balance criteria. The Linear Inverted Pendulum Model (LIPM) [2, 3], its multi-mass extensions [4], the Zero-Moment Point (ZMP) criterion [5], Capture Point and Divergent Component of Motion (DCM) methods [6–8], and full-body optimal control with state estimation [9, 10] have all produced stable walking on real hardware, but each requires substantial model-specific engineering.

Deep reinforcement learning offers a complementary, model-light perspective: rather than hand-designing a controller, an agent learns a control policy by interacting with a simulated environment and maximizing a reward [11–13]. RL has produced striking results in legged locomotion — for bipeds [14–16], for quadrupeds [17–19], in rich simulated environments [20], and through hierarchical skill learning [21] — but applying it to a full-size humanoid is difficult precisely because of the large action space and the fragility of balance.

This thesis investigates RL-based walking control for *Surena IV*, a full-size adult humanoid developed at the Center of Advanced Systems and Technologies (CAST) at the University of Tehran [22]. The central question is whether injecting classical-control structure into the RL problem can make learning tractable by shrinking the action space, rather than asking a network to discover joint-level coordination from scratch.

Contributions.

1. A flexible RL environment for Surena IV (the *direct* environment) in which the policy commands the robot's motors, designed so that action type (torque / position / velocity) and the number of actuated degrees of freedom can be swapped to compare approaches under a common reward.

2. A *hybrid* environment in which the policy outputs increments to the CoM and swing-foot trajectories (generated with an inverted-pendulum walking model) and a closed-form inverse-kinematics block produces the joint angles — reducing the action space from per-joint commands to a small set of trajectory parameters. This includes a fix to reconcile the classical method's reference frames with the simulator's feedback, and a contact-force-based scheme for stance/swing phase transitions. This is the thesis's primary methodological idea.

3. An empirical study comparing candidate RL algorithms (PPO vs. DDPG, TD3, SAC), a multi-term reward design grounded in walking biomechanics and the ZMP balance criterion, and an honest report of outcomes — including the agent exploiting an under-specified reward by walking backward, and the difficulty of recovering from near-unstable states.

2 Background

2.1 Reinforcement learning

In the standard RL setting the problem is framed as a Markov Decision Process: at each step the agent observes a state s, takes an action a under a policy $\pi(a \mid s)$, receives a reward $r(s, a)$, and transitions to a new state [11, 23]. The objective is to maximize the expected discounted return. For continuous-control problems such as locomotion, policy-gradient and actor–critic methods are

standard, including TRPO [24], DDPG, SAC, and PPO [25]. Benchmarking studies show that algorithm choice and implementation details matter substantially for continuous control [26,27].

2.2 Why PPO

PPO [25] optimizes a clipped surrogate objective that keeps each policy update close to the previous policy, trading a little theoretical rigor for robustness and ease of tuning. After comparing DDPG, TD3, and SAC on this problem, PPO was selected for its stability on high-dimensional continuous-action systems and its strong empirical track record on locomotion. The clipped objective is

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(\rho_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(\rho_t(\theta),\,1-\epsilon,\,1+\epsilon)\,\hat{A}_t\big)\Big], \qquad \rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, \tag{1}$$

where $\hat{A}_t$ is the estimated advantage and ϵ the clipping range.

2.3 Classical balance: the Zero-Moment Point

The ZMP is the point on the ground where the net moment of the inertial and gravitational forces has no horizontal component [5,28]. A walking motion is dynamically balanced as long as the ZMP remains inside the support polygon (the convex hull of the contact points). In this work the ZMP is computed from the measured foot-contact forces and used both as a balance check and as a reward signal in the hybrid environment.

3 Robot and Simulation Setup

The study targets the lower body of Surena IV [22]. Each leg has six actuated degrees of freedom — hip yaw, hip roll, hip pitch, knee pitch, ankle pitch, and ankle roll — for twelve lower-body DOF in total. Because the two hip-yaw joints contribute little to straight-line walking, the action space can be reduced to ten or fewer DOF when desired.

The robot is simulated in **PyBullet** [29], imported from a URDF model exported from the CAD description of the robot. The RL code is written in Python on top of the OpenAI Gym interface, with policies implemented in PyTorch [30] and trained using PPO from the Stable-Baselines library. Hyperparameters were tuned with Optuna.

4 Method

Two environments were developed, embodying two distinct philosophies for applying RL to humanoid walking.

4.1 Environment 1: direct motor control (baseline testbed)

In the first environment the action produced by the actor network is interpreted directly as a command to the robot's motors. The environment is deliberately built as a configurable testbed: a single parameter switches the meaning of the action between joint *torque*, joint *position*, and joint *velocity* commands, and the number of actuated DOF can be varied (e.g. 4, 8, 10, or 12) to study how problem dimensionality affects learning. The network outputs lie in $[-1, +1]$ and are denormalized to each actuator's command range; the joint-angle limits are taken from the URDF, and any command that would exceed the robot's working range is clipped to the corresponding floor/ceiling value.

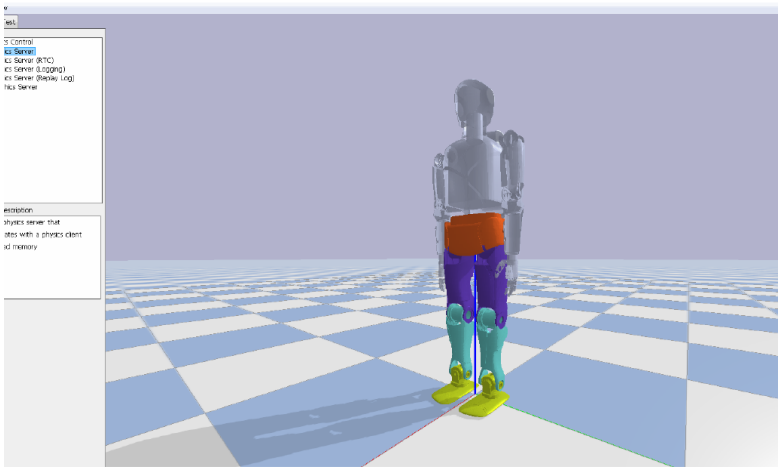


Figure 1: The Surena IV humanoid robot in the PyBullet simulation environment, imported from its URDF model. The thesis controls the twelve lower-body degrees of freedom (six per leg).

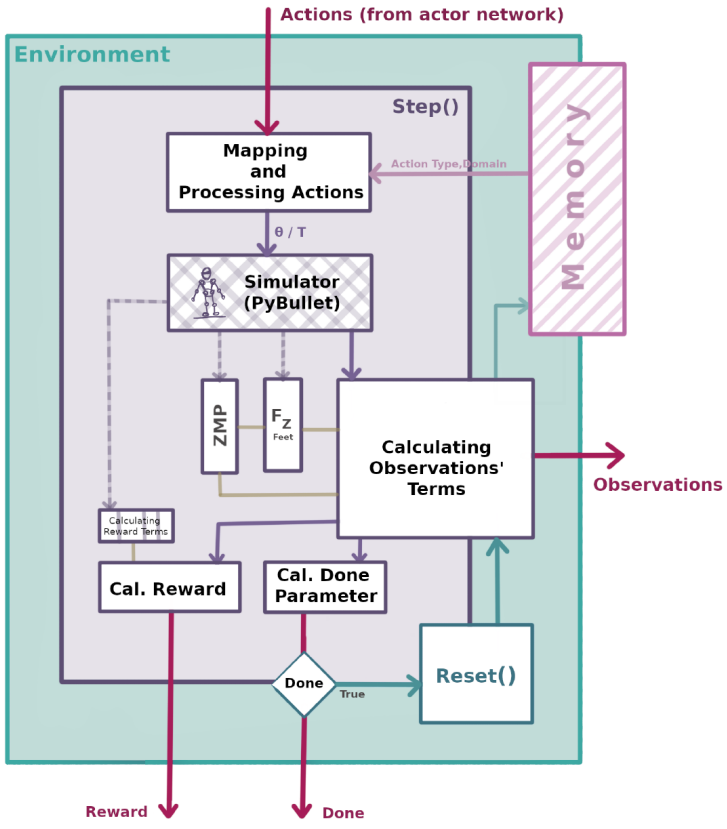


Figure 2: Architecture of the baseline (Environment 1) `step()` function. Actions from the actor network are mapped and processed into motor commands (θ for position or τ for torque, selected via the action type held in memory) and applied directly in PyBullet; the simulator feedback yields the ZMP and foot contact forces (F_z), from which the observation terms, reward, and termination flag are computed. Note the contrast with the hybrid environment (Fig. 4): there is no trajectory-generation or inverse-kinematics block — the policy commands the joints directly.

Observation. The observation includes the joint states, the CoM position and velocity, the pelvis orientation, the forces under the feet, and support-polygon / ZMP information.

Reward. The reward is a weighted sum of interpretable terms drawn from walking biomechanics (thesis Table 4-5). Positive terms reward forward CoM position and velocity, time steps survived, forward foot progress, and appropriate foot placement; negative terms penalize motor power consumption $\sum_n |\tau_n \dot{\theta}_n|$, lateral deviation of the CoM, total joint angular velocity $\sum_n |\dot{\theta}_n|$, changes in upper-body orientation, and falling short of a minimum forward velocity $\big|\min(v_x - 0.12,\, 0)\big|$. Each weight reflects the relative importance of the term; a term can be switched off entirely by setting its weight to zero (which makes the environment convenient for ablation), and individual terms can be wrapped in shaping functions — for instance a natural logarithm or a saturating transform — to reshape how strongly the reward responds as the term grows.

Two of the terms are defined by their own sub-functions. The *foot-placement* reward encourages the swing foot to land flat and well-oriented:

$$r_{fp} = \begin{cases} \Delta[\text{foot\_orientation}] \cdot [1,\, 1,\, 0.25], & z_{\text{start}} < z_{\text{foot}} < \text{threshold} \\ 0, & \text{otherwise,} \end{cases} \qquad \text{threshold} = 0.05\, z_{\text{start}} = 0.038, \tag{2}$$

so it is active only while the foot is near the ground. An optional *imitation* term, in the spirit of motion-imitation methods [31, 32] and informed by related work on human-motion reconstruction and reference-trajectory generation [33], rewards closeness of the robot's joint angles to a reference human motion:

$$r_{\text{imit}} = \exp\Big(- \sum_i \big(\theta_{\text{observation},i} - \theta_{\text{source},i}\big)^2 \Big), \tag{3}$$

and is gated by a switch variable that decides, per experiment, whether imitation is used at all. A representative reward used in experiments is

$$R(s,a) = 0.545\, v_x^2 + 1.25\, x + 0.2\, \max\big(x_{\text{left}}, x_{\text{right}}\big) - 0.45\, p_{\text{orientation}} + 0.55, \tag{4}$$

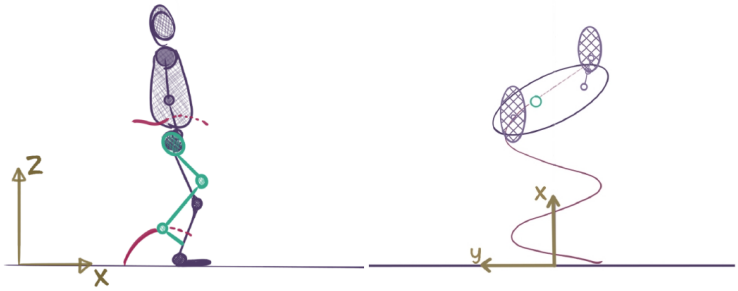
where v_x and x are the forward CoM velocity and position, $x_{\text{left}}, x_{\text{right}}$ are the forward foot positions, and $p_{\text{orientation}}$ penalizes upper-body tilt.

Termination, and a note on the balance criterion. An episode can be ended by either of two "fallen" criteria the thesis examines. The first, common in the RL literature, simply watches the *CoM height*: if it drops below a threshold the robot is taken to have fallen and the episode resets. The second, in the spirit of classical control, checks the *ZMP against the support polygon*: if the ZMP leaves the polygon the robot is treated as losing balance. A careful observation in the thesis is that these are not interchangeable — the ZMP-inside- support-polygon condition is *sufficient but not necessary* for staying upright [5]: a robot can momentarily lift the edge of its support foot (a tipping/roll transition) and still recover by changing its contact configuration, so a hard ZMP test would end episodes prematurely. For this reason the thesis prefers to use the ZMP not as a hard termination test but as a *soft* constraint inside the reward, penalizing ZMP excursions rather than forbidding them.

4.2 Environment 2: hybrid trajectory-level control (main contribution)

The second environment addresses the core difficulty of Environment 1: directly commanding ten or more joints gives the policy a large, weakly-structured action space in which coordinated walking

is hard to discover. The central idea of the thesis is to let the RL agent operate at the level of *trajectories* rather than joints, generate the leg motion with classical walking machinery, and let analytic kinematics handle the conversion to joint angles. The agent therefore *shapes* the reference trajectories that a classical controller would normally hand-design, and only those few trajectory parameters — not the joint commands — form the action space.



(b) Right-side view (a) Top view

Figure 3: Schematic of the quantities the agent shapes in the hybrid environment: the center-of-mass (CoM) trajectory and the swing-foot (ankle) trajectory, shown in right-side view (left) and top view (right). The lateral CoM motion follows an inverted-pendulum sway; the agent perturbs these reference paths.

Trajectory generation. The reference motion is built from two coupled trajectories. The *CoM trajectory* is generated using the inverted-pendulum walking model: the lateral (sideways) sway of the CoM is set relative to the current support foot, with a sway amplitude of $\pm 0.11380\,\mathrm{m}$ measured from the CoM (equivalently $\pm 2 \times 0.11380\,\mathrm{m}$ relative to the support foot's lateral position), while the nominal CoM height is held near $0.7\,\mathrm{m}$. The *swing-foot (ankle) trajectory* specifies, at each instant, the swing ankle's forward (x) and vertical (z) position so that the foot lifts, advances, and lands. All of these quantities are treated as adjustable: rather than fixing them as a classical planner would, the thesis lets the agent modify them online.

Action space. At each control step the actor network outputs small *increments* to the reference trajectories rather than absolute commands. The parameters needed to define the motion at a given instant are the CoM's x and y position and the swing ankle's x and z position; the agent's actions are increments $[\Delta x, \Delta y, \Delta z]$ applied to these, so the policy nudges the CoM and swing-foot paths step by step. Because absolute positions cannot be recovered from the agent's increments alone, the current measured CoM and ankle positions are fed back from the simulator and used as the base onto which the learned increments are added.

Inverse kinematics. Given the desired pelvis pose and the desired swing- and stance-ankle poses at an instant, a closed-form inverse-kinematics (IK) algorithm for the Surena IV leg [22] returns the joint angles. The leg is treated as a serial fixed-base chain under the assumption that the stance ankle is fixed: the linear position of the hip is determined by the ankle roll/pitch joints and the knee pitch, while the angular pose of the pelvis is affected by all joints, including hip roll, pitch, and yaw. The computation is run separately for the right and left legs. It uses the parameters of Table 1: the pelvis position and orientation (P_1, R_1), the foot position and orientation (P_f, R_f), and the link lengths L_h (thigh), L_p (pelvis), and L_{sh} (shank).

Table 1: Parameters of the closed-form inverse-kinematics computation for the Surena IV leg (thesis Table 4-6).

Symbol	Quantity	Role
P_1	Pelvis position	input pose
R_1	Pelvis orientation	input pose
P_f	Foot position	input pose
R_f	Foot orientation	input pose
L_h	Thigh link length	robot geometry
L_p	Pelvis link length	robot geometry
L_{sh}	Shank link length	robot geometry

Reconciling reference and feedback frames. A subtlety arises because the classical trajectory generator and the simulator do not refer to the same physical points. The inverted-pendulum reference assumes the ankle *link* as the foot reference and an idealized CoM, whereas PyBullet reports the ankle link's own center of mass and the pelvis CoM. The thesis applies a fixed translation (offset) vector that maps the assumed ankle/CoM locations onto the simulator's reported frames, ensuring the feedback used to update the trajectories is consistent with the classical method's assumptions. The thesis compares the commanded-vs.-feedback trajectories before this correction (Fig. 4-9 in the thesis) and after it; after correction the simulator feedback closely tracks the commanded trajectories, as shown in Fig. 5.

Phase transitions from contact forces. The stance/swing assignment and the start of the double-support phase are decided from the measured contact forces on the soles. When the force under the swing foot rises over several consecutive timesteps, it indicates that this foot has reached the ground and can take over as the support foot; the swing-foot indicator variable is then swapped at the next step, with the transition made gradually so that for a few timesteps both feet are on the ground before roles switch. The ZMP, computed from the same contact forces, is checked against the support polygon as a balance criterion.

This design reduces the action space from ten-plus per-joint commands to the small set of trajectory increments, while keeping the expressive balance machinery of classical walking (inverted-pendulum CoM generation, swing-foot trajectories, and ZMP monitoring) inside the loop. It is an instance of the broader principle — now common in the field under names such as residual and structured RL — of not asking a network to relearn what kinematics already provides.

Observation and reward. The observation (thesis Table 4-8) is, in order: the previous step's actions (n values); the current CoM position (3); the CoM position at the two previous steps (6); the current right-ankle position (3) and its two previous values (6); the current left-ankle position (3) and its two previous values (6); the vertical contact forces under the two feet (2); and the ZMP status (1). The reward (thesis Table 4-9) reuses the biomechanically-motivated terms of Environment 1 — rewarding forward CoM velocity ($\dot{x}^2$) and position, and survival time; penalizing motor power $\sum_n |\tau_n \dot{\theta}_n|$, lateral CoM deviation, total joint angular velocity $\sum_n |\dot{\theta}_n|$, and upper-body orientation change — and adds a positive term for keeping the ZMP within the support polygon.

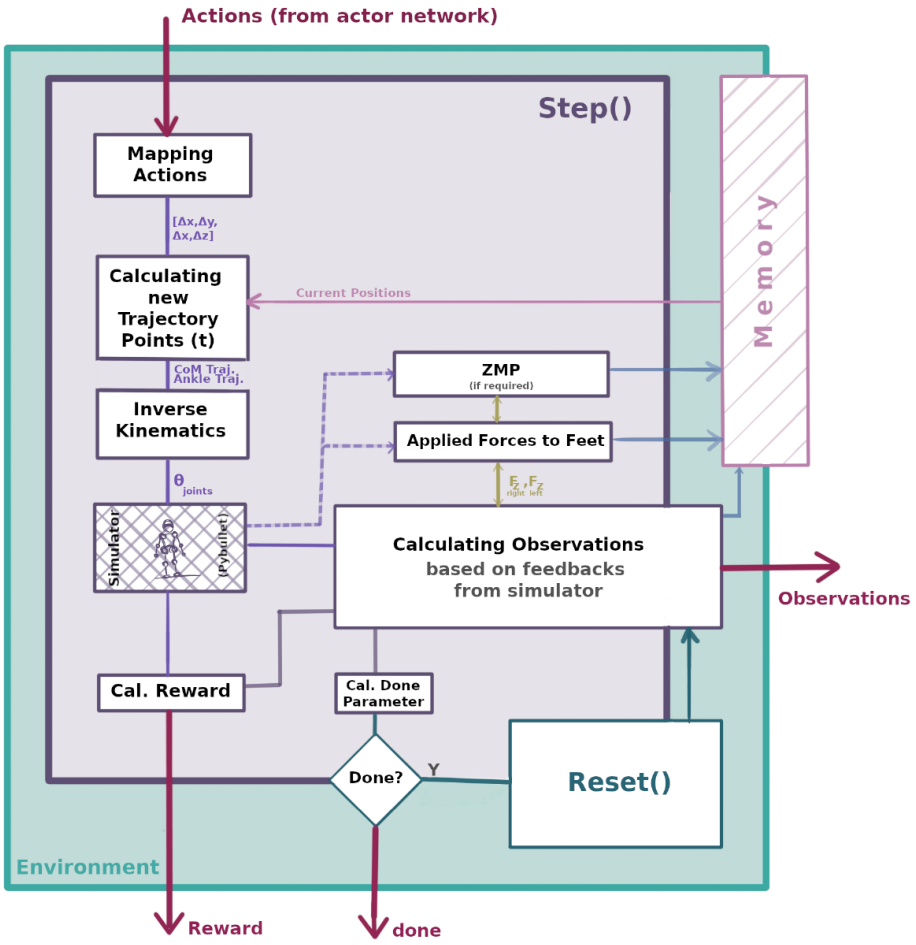
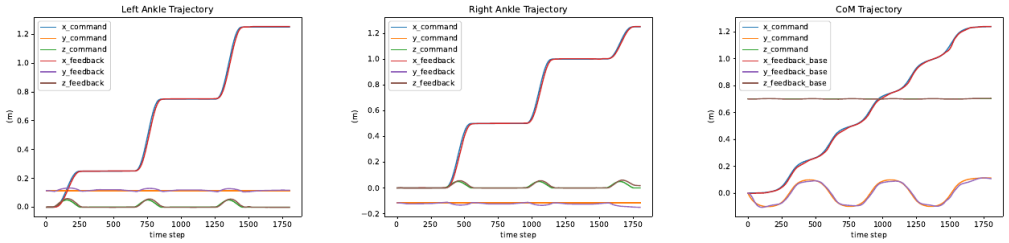


Figure 4: Architecture of the hybrid environment's `step()` function. The actions from the actor network are mapped to CoM/ankle trajectory increments $[\Delta x, \Delta y, \Delta z]$; new trajectory points are computed using the current positions held in memory; an inverse-kinematics block converts the desired poses to joint angles θ_{joints}, which are executed in PyBullet. Feedback (applied forces to the feet, and the ZMP if required) is used to compute the observation, reward, and termination flag, after which the environment either continues or resets. This trajectory-level parameterization is what reduces the action space relative to direct joint control.



(c) Left ankle (b) Right ankle (a) CoM and robot base

Figure 5: Commanded vs. achieved (feedback) trajectories for the CoM (right), the right ankle (center), and the left ankle (left). Close tracking confirms that the inverse-kinematics block faithfully realizes the trajectories the agent requests.

5 Experiments and Results

Training was performed with PPO under the configurations described above. Across runs, the episode-mean reward rose steadily, and the qualitative learning progression was consistent: the agent first learned to keep the robot balanced and standing, then to initiate forward walking. The two approaches reached different outcomes. The first (direct motor control) produced motion that kept the robot balanced throughout the entire episode, with no constraint imposed on the shape of the gait. The second (hybrid trajectory-level control) was successful up to roughly two forward steps — about 70 cm of forward travel — before the forward-motion terms ceased to be effective enough to carry the robot further.

Figure 6: Representative learning curve (episode-mean reward vs. environment time steps) for a position-control configuration. Reward increases as the policy learns to balance and step forward.

Reward exploitation. In a configuration whose reward emphasized only forward CoM motion, the agent discovered that it could keep the robot balanced more easily by moving *backward* rather than forward — an instance of reward exploitation. This is reported as a finding: it illustrates how sensitive learned locomotion is to reward specification, and motivated the additional foot-placement and orientation terms in the final reward.

Principal limitation. In all runs the reward improved as expected and the robot learned to balance and begin walking, but the central difficulty is recovery: once the robot enters a near-unstable state, returning from instability back to balance is hard for the agent to learn. The thesis

identifies this recovery problem as the key direction for future work. No hardware transfer was attempted; the sim-to-real gap for full-size humanoids remains substantial and is left to subsequent research.

6 Discussion and Outlook

The most durable idea in this thesis is the hybrid, trajectory-level parameterization of Environment 2. Reducing the action space by combining a learned policy with analytic kinematics and a classical balance criterion is a direction the field has continued to validate: layering RL on top of model-based controllers, feature-based controllers [34], or trajectory generators is now a standard recipe for tractable, sample-efficient legged learning. Viewed from 2022, when the dominant trend was end-to-end joint-level RL, choosing to embed structure was a reasonable and forward-looking design decision.

Promising next steps, several of which are now well-developed in the literature, include: an explicit push-recovery / stabilization objective and curriculum to address the recovery problem; domain randomization and system identification to enable sim-to-real transfer to the physical Surena IV; and reference-motion imitation [31] to shape natural gait while retaining the small trajectory-level action space.

Acknowledgments

This work was carried out at the Center of Advanced Systems and Technologies (CAST), University of Tehran, under the supervision of Dr. Aghil Yousefi-Koma and Dr. Masoud Shariat-Panahi. The author thanks the CAST humanoid team for the Surena IV platform and model.

References

[1] H. Miura and I. Shimoyama, "Dynamic walk of a biped," *The International Journal of Robotics Research*, vol. 3, no. 2, pp. 60–74, 1984.

[2] S. Kajita, F. Kanehiro, K. Kaneko, K. Yokoi, and H. Hirukawa, "The 3D linear inverted pendulum mode: A simple modeling for a biped walking pattern generation," in *Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems*, vol. 1, pp. 239–246, IEEE, 2001.

[3] S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Yokoi, and H. Hirukawa, "Biped walking pattern generation by a simple three-dimensional inverted pendulum model," *Advanced Robotics*, vol. 17, no. 2, pp. 131–147, 2003.

[4] M. Eslami, A. Yousefi-Koma, and M. Khadiv, "A novel three-mass inverted pendulum model for real-time trajectory generation of biped robots," in *2016 4th International Conference on Robotics and Mechatronics (ICROM)*, pp. 404–409, IEEE, 2016.

[5] M. Vukobratović and B. Borovac, "Zero-moment point—thirty five years of its life," *International Journal of Humanoid Robotics*, vol. 1, no. 1, pp. 157–173, 2004.

[6] J. Pratt, J. Carff, S. Drakunov, and A. Goswami, "Capture point: A step toward humanoid push recovery," in *2006 6th IEEE-RAS International Conference on Humanoid Robots*, pp. 200–207, IEEE, 2006.

[7] J. Englsberger, C. Ott, M. A. Roa, A. Albu-Schäffer, and G. Hirzinger, "Bipedal walking control based on capture point dynamics," in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 4420–4427, IEEE, 2011.

[8] A. Vedadi, K. Sinaei, P. Abdolahnezhad, S. S. Aboumasoudi, and A. Yousefi-Koma, "Bipedal locomotion optimization by exploitation of the full dynamics in DCM trajectory planning," in *2021 9th RSI International Conference on Robotics and Mechatronics (ICRoM)*, pp. 365–370, IEEE, 2021.

[9] K. Ishihara, T. D. Itoh, and J. Morimoto, "Full-body optimal control toward versatile and agile behaviors in a humanoid robot," *IEEE Robotics and Automation Letters*, vol. 5, no. 1, pp. 119–126, 2019.

[10] X. Xinjilefu, S. Feng, W. Huang, and C. G. Atkeson, "Decoupled state estimation for humanoids using full-body dynamics," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 195–201, IEEE, 2014.

[11] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT Press, 2018.

[12] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau, "An introduction to deep reinforcement learning," *arXiv preprint arXiv:1811.12560*, 2018.

[13] T. Mitchell, *Machine learning*. McGraw-Hill, 1997.

[14] J. Morimoto and C. G. Atkeson, "Learning biped locomotion," *IEEE Robotics & Automation Magazine*, vol. 14, no. 2, pp. 41–51, 2007.

[15] Z. Li, X. Cheng, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath, "Reinforcement learning for robust parameterized locomotion control of bipedal robots," *arXiv preprint arXiv:2103.14295*, 2021.

[16] C. Yang, K. Yuan, S. Heng, T. Komura, and Z. Li, "Learning natural locomotion behaviors for humanoid robots using human bias," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2610–2617, 2020.

[17] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.

[18] C. Gehring, S. Coros, M. Hutter, C. D. Bellicoso, H. Heijnen, R. Diethelm, M. Bloesch, P. Fankhauser, J. Hwangbo, M. Hoepflinger, *et al.*, "Practice makes perfect: An optimization-based approach to controlling agile motions for a quadruped robot," *IEEE Robotics & Automation Magazine*, vol. 23, no. 1, pp. 34–43, 2016.

[19] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, "MIT cheetah 3: Design and control of a robust, dynamic quadruped robot," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2245–2252, IEEE, 2018.

[20] N. Heess, D. TB, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. M. Eslami, *et al.*, "Emergence of locomotion behaviours in rich environments," *arXiv preprint arXiv:1707.02286*, 2017.

[21] T. Li, N. Lambert, R. Calandra, F. Meier, and A. Rai, "Learning generalizable locomotion skills with hierarchical reinforcement learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 413–419, IEEE, 2020.

[22] A. Yousefi-Koma, B. Maleki, H. Maleki, A. Amani, M. A. Bazrafshani, H. Keshavarz, A. Iranmanesh, A. Yazdanpanah, H. Alai, S. Salehi, *et al.*, "SurenaIV: Towards a cost-effective full-size humanoid robot for real-world scenarios," in *2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids)*, pp. 142–148, IEEE, 2021.

[23] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

[24] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International Conference on Machine Learning*, pp. 1889–1897, PMLR, 2015.

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[26] Y. Duan, X. Chen, R. Houthooft, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *International Conference on Machine Learning*, pp. 1329–1338, PMLR, 2016.

[27] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, 2018. arXiv:1709.06560.

[28] S. Kajita, H. Hirukawa, K. Harada, and K. Yokoi, *Introduction to humanoid robotics*, vol. 101. Springer, 2014.

[29] E. Coumans and Y. Bai, "PyBullet quickstart guide." `https://pybullet.org`, 2016.

[30] M. Ibrahim, "PyTorch vs TensorFlow 2021," Jun 2021.

[31] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "DeepMimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 4, pp. 1–14, 2018.

[32] X. B. Peng, E. Coumans, T. Zhang, T.-W. Lee, J. Tan, and S. Levine, "Learning agile robotic locomotion skills by imitating animals," *arXiv preprint arXiv:2004.00784*, 2020.

[33] M. Malekzadeh Arasteh, "Reconstruction and modeling of human motion and trajectory generation for a robot," Master's thesis, University of Tehran, Faculty of Mechanical Engineering, 2020. In Persian.

[34] M. De Lasa, I. Mordatch, and A. Hertzmann, "Feature-based locomotion controllers," *ACM Transactions on Graphics (TOG)*, vol. 29, no. 4, pp. 1–10, 2010.

Appendix: Supplementary Material

This appendix collects additional figures and details from the original thesis that did not fit the main summary but may be useful to readers who want a closer look at the setup, the observation space, the reward variants explored, and the training behaviour.

Code and original thesis. The implementation — the custom OpenAI Gym environments for Surena IV, the trajectory generation, inverse kinematics, and training scripts — is available at `github.com/RHnejad/gym-Surena`. The full original (Persian) thesis is available here.

A.1 Background: walking phases, the inverted pendulum, PPO, and the ZMP

For readers less familiar with bipedal locomotion or with PPO, four figures from the thesis ground the rest of the work. Human walking is cyclic (Fig. 7): a *stance* phase (heel strike → foot flat → midstance → heel off → toe off), during which the foot supports the body, and a *swing* phase (toe off → midswing → next heel strike), during which the foot is in the air and advancing. The hybrid environment's swing-foot trajectory is exactly the motion of the leg through this swing phase.

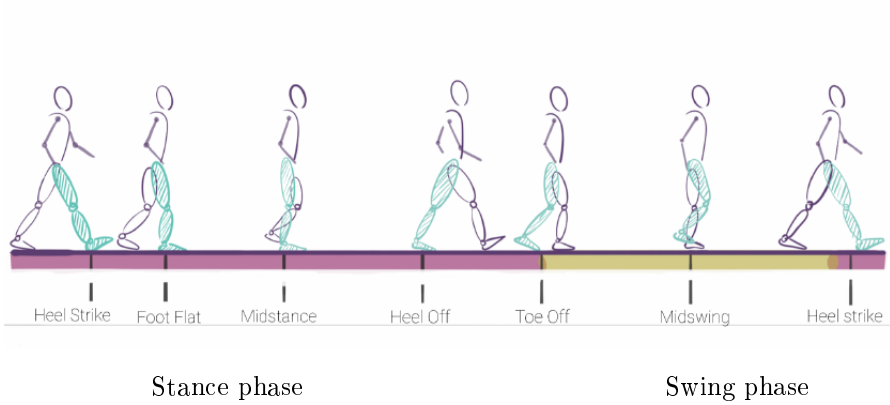


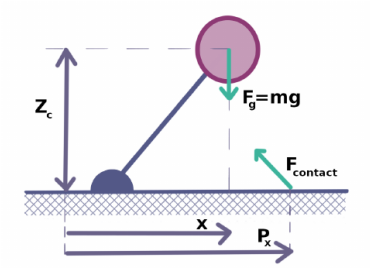
Figure 7: The human gait cycle (thesis Fig. 2-1): the stance phase (left) and swing phase (right), with the canonical gait events labelled.

Classical bipedal controllers approximate the robot as a *linear inverted pendulum* (Fig. 8a): a point mass at height Z_c above a ground contact point, under gravity $F_g = mg$ and a ground contact force. This reduced model gives the center-of-mass dynamics that the hybrid environment's CoM trajectory is built from; the same family of models underlies Capture Point and Divergent Component of Motion (DCM) planning [2, 6].

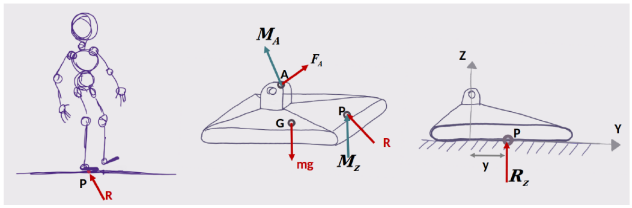
Finally, the chosen learning algorithm, PPO [25], optimizes a *clipped* surrogate objective (Fig. 9). The clip caps how far a single update can move the policy's probability ratio r away from 1, which is what gives PPO its stability on high-dimensional continuous-control problems like this one — and, as the learning curves below show, what keeps training from diverging even when individual episodes end abruptly.

A.2 The observation space

Figure 10 shows every channel the agent observes over one episode in the direct-control environment: the twelve joint angles θ_0–θ_{11} and their velocities, the CoM position and velocity, the pelvis



(a) Linear inverted-pendulum model (thesis Fig. 3-2).



(b) Foot free-body diagram for the zero-moment point (thesis Fig. 4-6).

Figure 8: The classical machinery the hybrid approach builds on: a reduced-order CoM model (left) and the zero-moment-point balance geometry (right). The ZMP is the ground point P about which the horizontal moment of the contact reaction vanishes; staying inside the support polygon is a sufficient condition for not tipping.

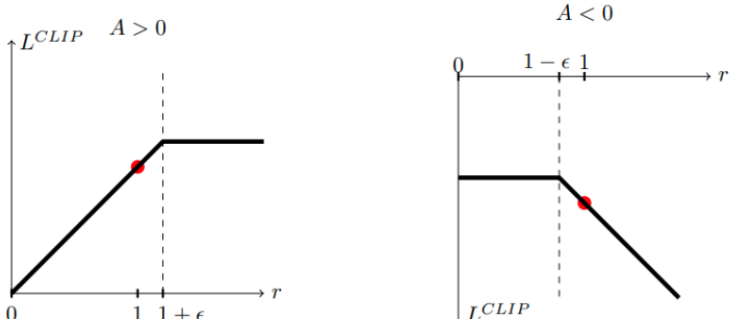


Figure 9: PPO's clipped objective L^{CLIP} as a function of the probability ratio r, for positive ($A > 0$) and negative ($A < 0$) advantage (thesis Fig. 2-3). The flat regions are where the clip removes the incentive to push r further.

orientation (as a quaternion) and angular velocity, and the vertical contact force F_z under each foot. The foot-force traces (channels 36–37) make the gait cycle visible — the alternating loading of the two feet — and the zero-moment point used for the balance criterion is computed from these contact forces via

$$(\vec{OP} \times \vec{R})^H + \vec{OG} \times m_s g + M_A^H + (\vec{OA} \times F_A)^H = 0. \tag{5}$$

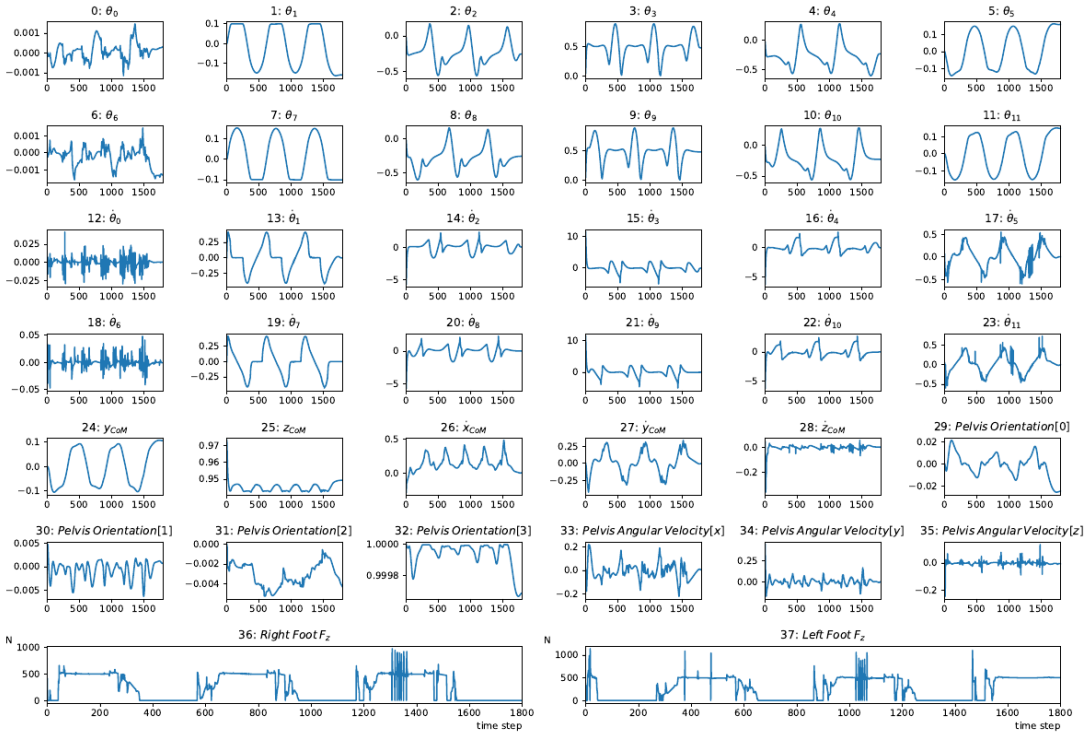


Figure 10: The full observation vector over one episode (thesis Fig. 4-4): joint angles and velocities, CoM position and velocity, pelvis orientation and angular velocity, and per-foot vertical contact forces.

A.3 The robot walking

Figure 11 shows a sequence of frames from a position-control training episode: the robot maintains balance and advances. Figure 12 shows the torque profiles at all twelve actuators during a walking sequence.

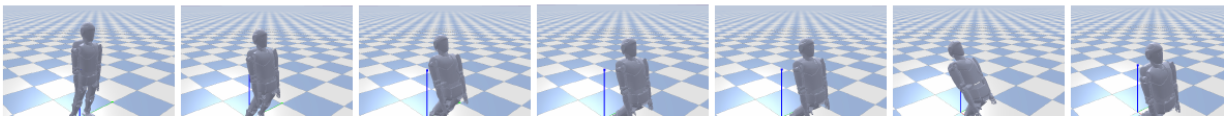


Figure 11: Frames from a position-control training episode (thesis Fig. 5-6): the robot keeps its balance while moving.

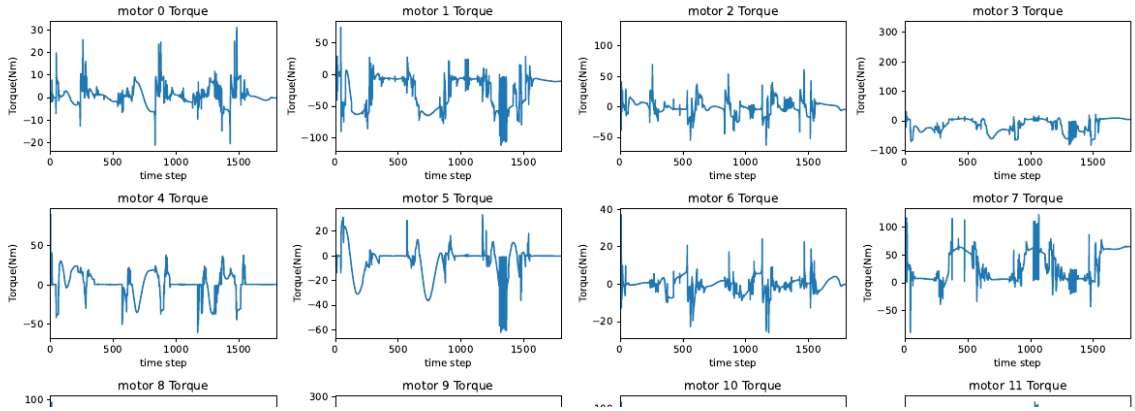
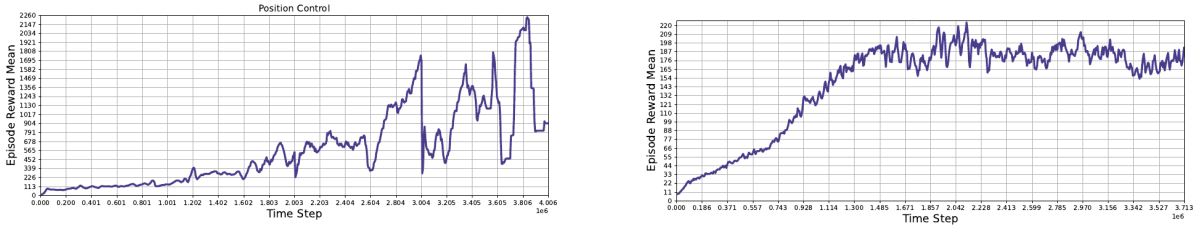


Figure 12: Per-actuator torque profiles for the twelve lower-body motors during a walking sequence (top eight of twelve shown; thesis Fig. 5-13).

A.4 Learning curves: position vs. torque control

The two action types behave differently during training (Fig. 13). Under *position* control the episode-mean reward climbs with occasional sharp drops — moments where some episodes ended early — but PPO's clipped update keeps the policy from diverging, so the overall trend stays upward. Under *torque* control the reward converges sooner and then plateaus, improving little afterward.



(a) Position control **(b)** Torque control

Figure 13: Episode-mean reward vs. environment time steps for the two action types (thesis Figs. 5-2 and 5-3).

A.5 Reward-function variants explored

The reward was refined across experiments. Beyond the representative form given in the main text, two further variants illustrate the progression. A purely progress-and-orientation form,

$$R(s,a) = 2.5\,x - 0.9\sum_{n=1}^{4}\big|\text{Curr.Or.}[n] - \text{Strt.Or.}[n]\big| + 0.65, \tag{6}$$

adds an explicit penalty on upper-body orientation change (in quaternion coordinates) to discourage the robot from twisting its torso to chase the forward-progress reward. The foot-placement and minimum-velocity sub-rewards (Section 4) were then introduced to further shape the gait. Each variant corresponds to a different hyperparameter configuration; representative values appear in Table 2.

Table 2: Representative training configurations across experiments (from thesis Tables 4-4, 5-1–5-6). "Env" is the number of parallel environments.

Setting	Learning rate	Env	Actor/critic hidden layers	Total steps
Position control	0.002	4	128/64/64	4.0×10^6
Forward-motion reward	0.0015	4	64/64/32	1.4×10^6
Orientation term	0.00015	4	64/64/32	3.0×10^6
Actor–critic study	0.0003	6	64/64	3.9×10^6

PPO was trained on top of the Stable-Baselines implementation with parallel environments across CPU cores; common settings included a discount factor $\gamma = 0.99$, a clipping range $\epsilon = 0.2$, a batch size of 64, and 10 optimization epochs per update, with hyperparameters tuned using Optuna.